Route Prefix in ASP.NET Core Web API

Back to: ASP.NET Core Web API Tutorials

Route Prefix in ASP.NET Core Web API

In this article, I will discuss **Route Prefix in ASP.NET Core Web API Routing** with Examples. Please read our previous article about **Token Replacement in ASP.NET Core Web API**.

In ASP.NET Core Web API, a **Route Prefix** defines a **Common Path Segment** that applies to **All Routes within a Controller**. It allows you to avoid repeating the same route segment on every action method, resulting in cleaner, shorter, and more maintainable route definitions. Let's understand the **concept, usage, and override mechanism** of the Route Prefix in ASP.NET Core Web API with a real-world example.

Understanding the Need for a Route Prefix in ASP.NET Core Web API

When we define routes at the **action method level**, it's common for multiple actions within a controller to share a **common route prefix**. For example, in the EmployeeController, every route begins with the word **employee**. Let's start with a simple implementation **without using a route prefix**:

Without a route prefix, you have to manually include this common segment in each action method, which leads to **duplication**. First, modify the EmployeeController class, as shown below.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [Route("employee/all")]
        [HttpGet]
        public string GetAllEmployees()
        {
            return "Response from GetAllEmployees Method";
        }

        [Route("employee/{Id}")]
        [HttpGet]
        public string GetEmployeeById(int Id)
        {
            return $"Response from GetEmployeeById Method, Id : {Id}";
        }
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        [Route("employee/department/{Department}")]
        [HttpGet]
        public string GetDepartmentEmployees(string Department)
        {
            return $"Response from GetDepartmentEmployees Method, Department :
{Department}";
        }
    }
}
```

Code Explanation

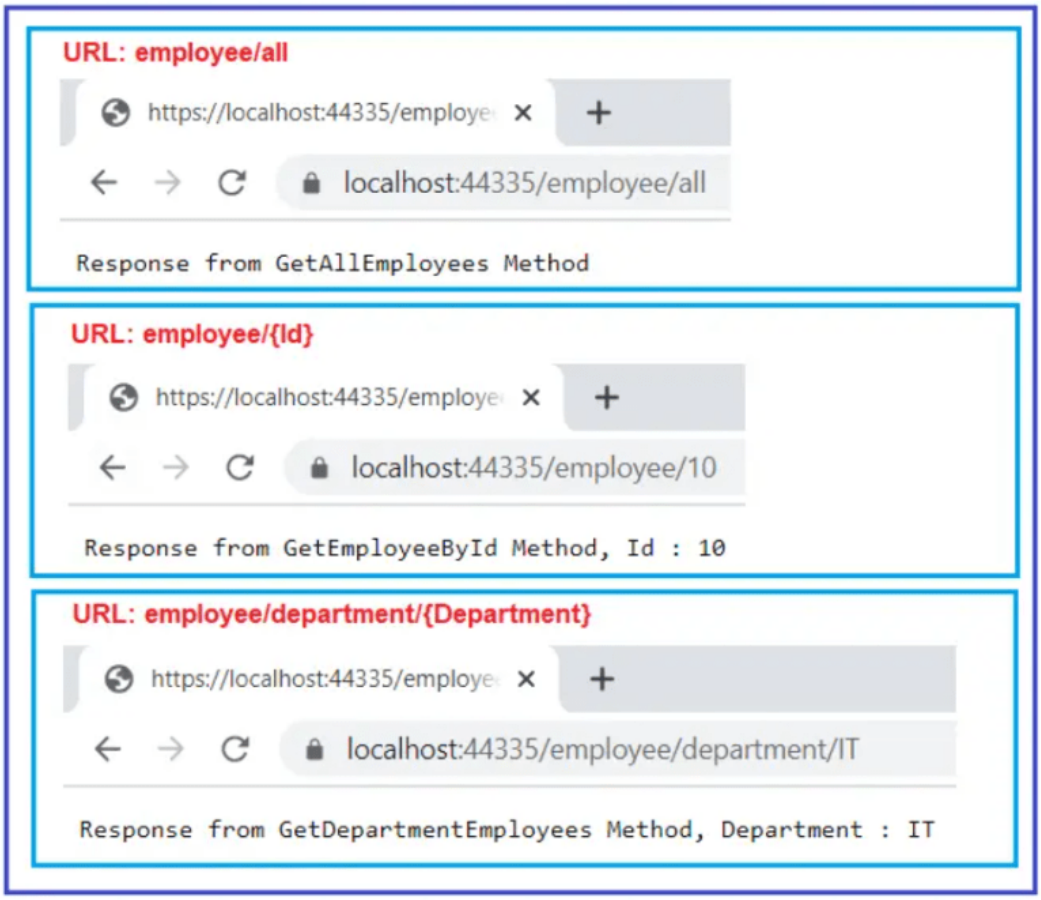
- Each [Route] attribute explicitly includes the employee prefix.
- If you later change the controller's scope (e.g., from employee to staff), you'll need to update every route manually.
- This approach works, but it is **not efficient or scalable**.

All these routes work correctly:

- /employee/all
- /employee/1
- /employee/department/HR

However, repeating the "employee" prefix for every action is inefficient. Now, you can access the three methods above, as shown in the image below.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Real-time Analogy: It's like writing your company name on every single document manually instead of printing it once on the letterhead.

Defining a Route Prefix at the Controller Level in ASP.NET Core Web API

To simplify routing, we can move the **common prefix** to the **controller level**. This ensures every action automatically includes the same prefix in its URL, while each method only defines the part unique to it.

The route prefix is typically defined using the [Route] attribute at the controller level. This attribute specifies the initial segment(s) of the URI that will be common to all actions within that controller. So, please modify the Employee Controller as follows:

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("employee")]
    public class EmployeeController : ControllerBase
    {
        [Route("all")]
        [HttpGet]
        public string GetAllEmployees()
        {
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            return "Response from GetAllEmployees Method";
        }

        [Route("{Id}")]
        [HttpGet]
        public string GetEmployeeById(int Id)
        {
            return $"Response from GetEmployeeById Method, Id : {Id}";
        }

        [Route("department/{Department}")]
        [HttpGet]
        public string GetDepartmentEmployees(string Department)
        {
            return $"Response from GetDepartmentEmployees Method, Department :
{Department}";
        }
    }
}
```

Code Explanation

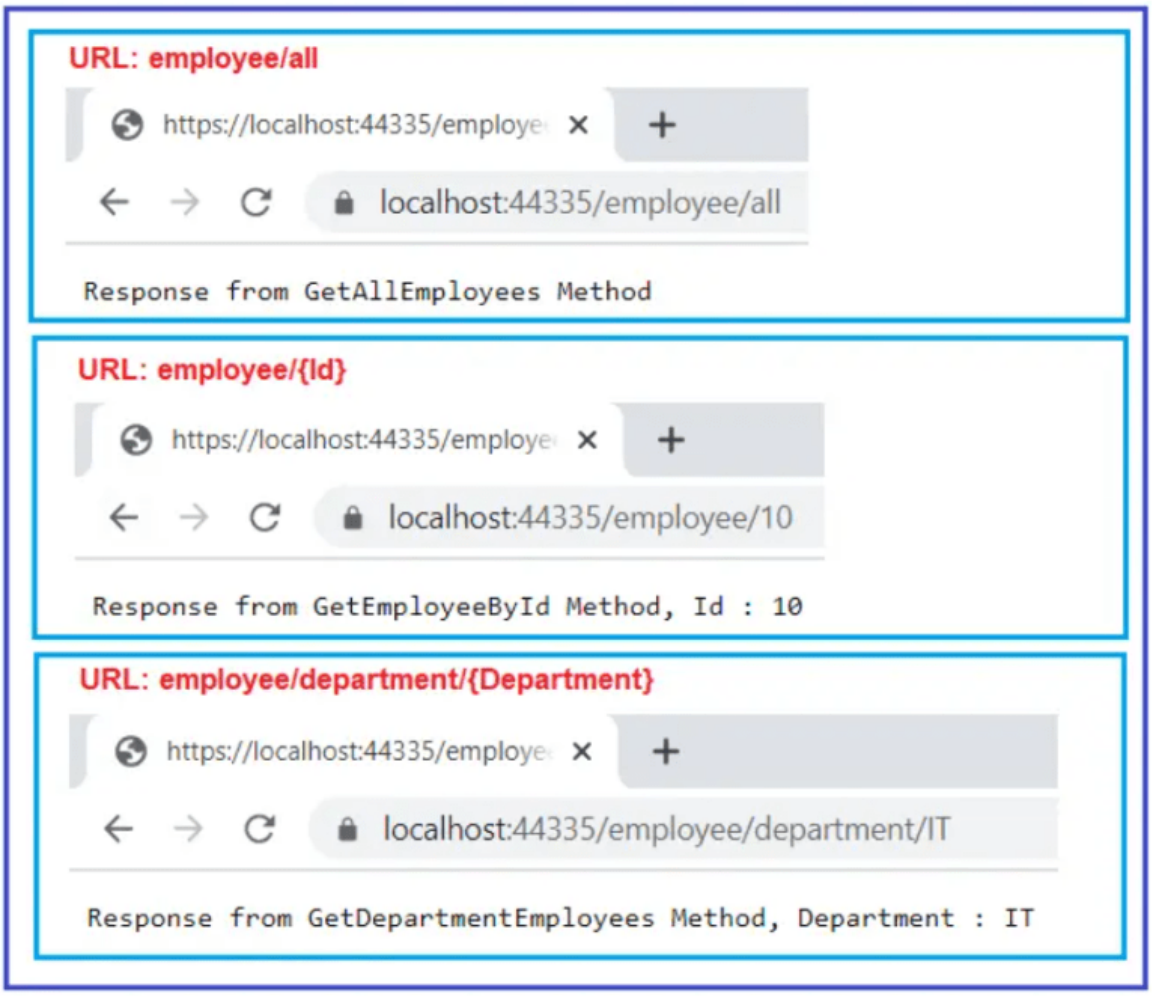
- The controller-level [Route("employee")] acts as a **prefix**.
- All actions within the controller automatically inherit this prefix.
- Each method now only defines the **specific part** of the URL relevant to its logic.

Resulting Endpoints

- /employee/all
- /employee/{Id}
- /employee/department/{Department}

With the above changes in place, you can access the above three resources as we accessed them in our previous example, as shown in the image below.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Analogy: It's like setting a **default folder path** for saving files; every new file automatically goes inside that folder unless you explicitly change it.

How Do We override the Route Prefix in ASP.NET Core Web API?

Sometimes, we may want **one or more actions** to have a route **outside** the controller's prefix. By default, if a route prefix exists at the controller level, all routes inside the controller inherit it. Let's see what happens if we add a new endpoint that doesn't follow the employee prefix.

Currently, our Employee Controller class contains three action methods, which start with the same Route Prefix, i.e., employee. We want to add one action method within the employee controller to return all departments. And we want to access this resource using the URL: **department/all**.

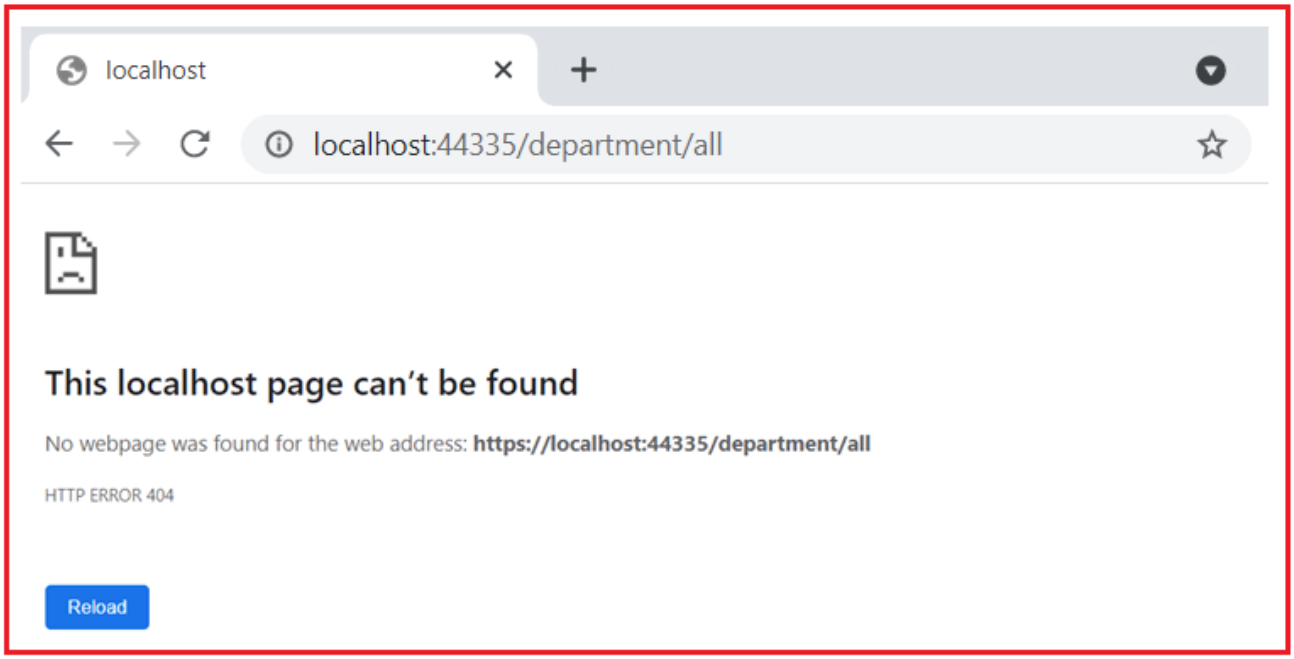
So, let us first add the following GetAllDepartment method and decorate it with [Route("department/all")] Attribute as shown below within the EmployeeController.

```
[Route("department/all")]
[HttpGet]
public string GetAllDepartment()
{
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    return "Response from GetAllDepartment Method";
}
```

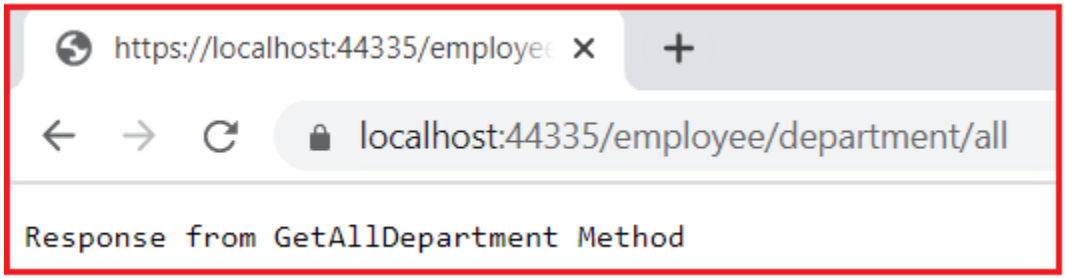
With the above **[Route("department/all")]** attribute on the GetAllDepartment() method, when we navigate to **department/all**, we will get the following error.



What Happens

- When you navigate to /department/all, it **does not work**.
- However, /employee/department/all **does work** as shown in the image below.

This happens because the [Route("employee")] prefix at the controller level is automatically applied to every action in the controller.



To fix this, you must **override** the controller-level route prefix.

Overriding a Controller-Level Route Prefix Using the Tilde (~) Symbol

To make an action **bypass** the controller's prefix, prefix the route with a **tilde (~)** character. So, please modify the GetAllDepartment action method as shown below to use the tilde symbol to override the route defined at the employee controller.

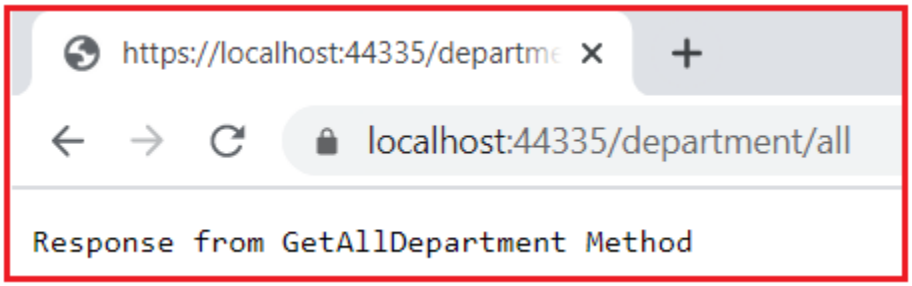
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
[Route("~/department/all")]
[HttpGet]
public string GetAllDepartment()
{
    return "Response from GetAllDepartment Method";
}
```

Code Explanation

- The ~ symbol tells ASP.NET Core to **ignore** the controller's route prefix (employee) and treat the route as **absolute**.
- Now, /department/all will map directly to the GetAllDepartment() method.

With the above change in place, the GetAllDepartment() action method is now mapped to the URI "**/department/all**" as expected, as shown in the image below.



Real-time Analogy: Think of the tilde (~) like taking a **shortcut road** that skips the main route prefix; you directly reach the endpoint you specify.

Using HTTP Method Attributes Instead of [Route]

Instead of using [Route] for every method, you can define routes directly within HTTP method attributes such as [HttpGet], [HttpPost], [HttpPut], and [HttpDelete]. This makes your controller even cleaner and more expressive. For a better understanding, please modify the Employee Controller as follows, then check each individual endpoint; it should work as expected.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("employee")]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("all")]
        public string GetAllEmployees()
        {
            return "Response from GetAllEmployees Method";
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        }

        [HttpGet("{Id}")]
        public string GetEmployeeById(int Id)
        {
            return $"Response from GetEmployeeById Method, Id : {Id}";
        }

        [HttpGet("department/{Department}")]
        public string GetDepartmentEmployees(string Department)
        {
            return $"Response from GetDepartmentEmployees Method, Department :
{Department}";
        }

        [HttpGet("~/department/all")]
        public string GetAllDepartment()
        {
            return "Response from GetAllDepartment Method";
        }
    }
}
```

Advantages of Using Route Prefix in ASP.NET Core Web API

1. **Reduces Repetition**: Avoids repeating the same prefix in multiple routes inside a controller.
2. **Improves Maintainability**: Changing a prefix (like employee to staff) requires editing only one line at the controller level.
3. **Provides Clarity and Structure**: Grouping related endpoints under a shared prefix clearly reflects the API's logical structure.
4. **Enables Consistent URL Patterns**: Promotes a uniform convention across all controllers, improving readability for clients and developers alike.
5. **Supports Overrides When Needed**: The ~ symbol gives flexibility to define exceptions when specific routes should not include the default prefix.

A **Route Prefix** in ASP.NET Core Web API simplifies and organizes routing by grouping related endpoints under a shared base path. It minimizes redundancy, enhances maintainability, and provides flexibility to override routes when necessary using the ~ symbol.

In the next article, I will discuss **Route Constraints in ASP.NET Core Web API** with Examples. In this article, I try to explain the **Route Prefix in ASP.NET Core Web API Attribute Routing** with examples. I hope you enjoy this Route Prefix in the ASP.NET Core Web API Attribute Routing article.

Privacy Preferences

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent Do Not Sell My Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF